SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE:** MLA0101

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

I Namburi Sai Yashaswini(192425250)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **yashaswini**

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	**Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.**	Informed Search, Greedy Best-First Search, Heuristic Functions
2	**A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate g(n), h(n), and f(n) for each expanded node, and explain why A guarantees the optimal solution.**	A* Search, Heuristic Functions
3	**A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.**	CSP, MRV, Forward Checking
4	**Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.**	Game Playing, Minimax, Alpha–Beta Pruning
5	**A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.**	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	**Problem Statement**	Briefly describe the given problem in your own words.
2	**Problem Analysis**	Identify the initial state, goal state, assumptions, and constraints.
3	**AI Technique Selection**	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	**Justification of Algorithm**	Explain why the selected algorithm is suitable compared to other search techniques.
5	**State Space Representation**	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	**Variables, Domains, and Constraints (Applicable for CSP problems)**	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	**Heuristic Function (Applicable for Informed Search)**	Define the heuristic function h(n) and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	**Algorithm Design / Pseudocode**	Write the step-by-step algorithm or pseudocode for solving the problem.
9	**Flowchart / Algorithm Diagram**	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	**Working / Execution Trace**	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	**Complexity Analysis**	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	**Advantages and Limitations**	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	**Conclusion**	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.
Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.

Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.

Contents

Q1 — Greedy Best-First Search: Graph Pathfinding

Q2 — A* Search: Delivery Robot Route Optimization

Q3 — Constraint Satisfaction Problem: Exam Timetabling

Q4 — Minimax & Alpha–Beta Pruning: Game Tree Analysis

Q5 — Online Search Agent / Local Search: Dynamic Warehouse Robot

QUESTION 1

Greedy Best-First Search — Graph Pathfinding

1. Problem Statement

Given a graph of nodes connected by paths, an agent must travel from a source node S to a goal node G. Each node is associated with an estimated distance to the goal, called the heuristic value h(n). The task is to design an algorithm that quickly finds a path from S to G by always moving toward the node that appears closest to the goal.

2. Problem Analysis

- Initial State: Source node S.
- Goal State: Node G is reached.
- State Space: Nodes {S, A, B, C, D, G} connected by undirected edges (graph shown in Section 5).
- Assumptions: A heuristic estimate h(n) is available at every node; the graph is static (does not change during search).
- Constraints: The agent can only move along existing edges and must select, at every step, the unvisited neighbour with the lowest h(n).

3. AI Technique Selection

Greedy Best-First Search (an Informed / Heuristic Search technique).

4. Justification of Algorithm

Greedy Best-First Search expands the node that appears closest to the goal, using only h(n), so it converges towards the goal much faster than uninformed methods such as BFS or DFS, which ignore goal information entirely. Compared to A* Search, it is computationally lighter because it does not track the accumulated path cost g(n) — making it well suited for problems such as quick real-time robot navigation where a fast, reasonable route matters more than a mathematically optimal one.

5. State Space Representation

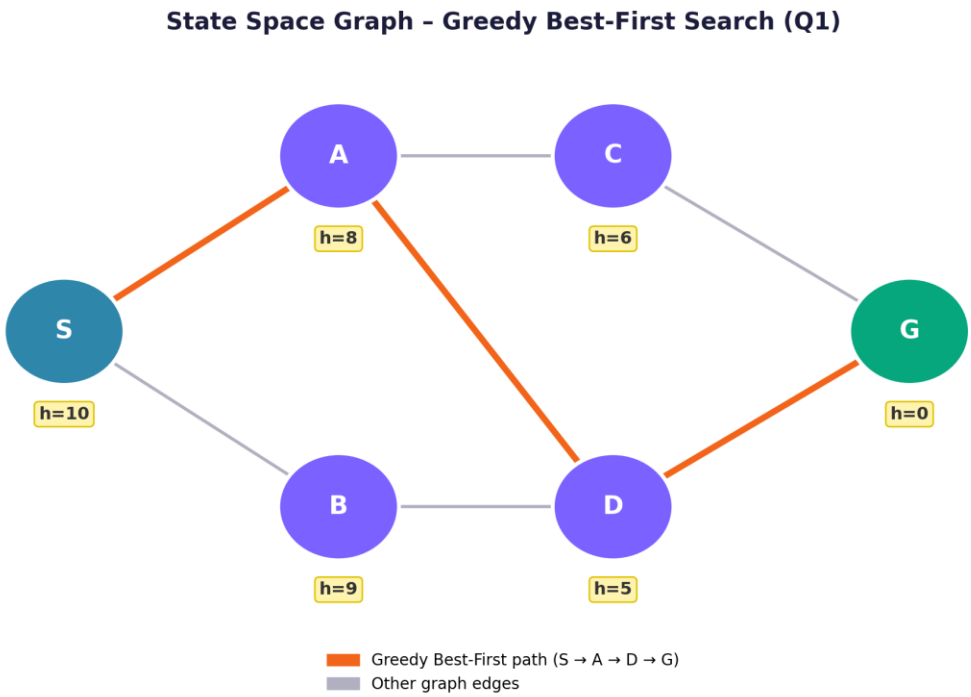

Fig. 1 — State-space graph with heuristic values h(n); the orange path shows the route chosen by Greedy Best-First Search.

6. Variables, Domains and Constraints (CSP)

Not Applicable — this is a graph-search problem, not a CSP.

7. Heuristic Function

h(n) = straight-line / estimated distance from node n to the goal node G (values shown under each node in Fig. 1). This heuristic is not guaranteed to be admissible; it is used purely to rank and order node expansion, which is characteristic of Greedy Best-First Search.

8. Algorithm Design / Pseudocode

```
function GREEDY-BEST-FIRST-SEARCH(start, goal):
    frontier <- PriorityQueue ordered by h(n)
    frontier.push(start, h(start))
    explored <- { }
    while frontier is not empty:
        n <- frontier.pop_min()
        if n == goal: return RECONSTRUCT-PATH(n)
        explored.add(n)
        for each neighbour m of n:
            if m not in explored and m not in frontier:
                frontier.push(m, h(m))
    return failure
```

9. Flowchart / Search Tree

Refer to Fig. 1 (Section 5) — the orange highlighted edges represent the search-tree path actually explored and taken by the algorithm.

10. Working / Execution Trace

Step	Node Expanded	h(n)	Frontier After Expansion
1	S	10	A(8), B(9)
2	A	8	D(5), C(6), B(9)
3	D	5	G(0), C(6), B(9)
4	G (goal)	0	— search stops —

Resulting path: S → A → D → G, chosen purely by always expanding the lowest-h(n) neighbour.

11. Complexity Analysis

- Time Complexity: O(b^m) in the worst case, where b = branching factor and m = maximum depth of the search space.
- Space Complexity: O(b^m), since all generated nodes are kept in the frontier/explored list.
- In practice, with an informative heuristic, far fewer nodes are expanded than the worst-case bound.

12. Advantages and Limitations

- Advantage: Very fast in practice; requires less bookkeeping than A* since path cost g(n) is not tracked.
- Advantage: Simple to implement using a priority queue keyed only on h(n).
- Limitation: Not optimal — the returned path (S-A-D-G) is not guaranteed to be the lowest-cost path.
- Limitation: Not complete in infinite or cyclic spaces without an explored/visited set.

13. Conclusion

Greedy Best-First Search efficiently guided the agent from S to G using only heuristic estimates, expanding just 3 nodes. Its speed comes at the cost of optimality; combining it with path-cost tracking — effectively upgrading it to A* Search — would guarantee the shortest path while retaining heuristic guidance.

QUESTION 2

A* Search — Delivery Robot Route Optimization

1. Problem Statement

A delivery robot must travel from a warehouse (S) to a customer location (G) through a weighted road network. Roads between locations have different travel costs, and the robot must find the least-cost path while still being guided efficiently by a heuristic estimate.

2. Problem Analysis

- Initial State: Warehouse node S.
- Goal State: Customer node G.
- State Space: Nodes {S, A, B, C, D, G}, edges are road segments each with a positive travel cost (Fig. 2).
- Assumptions: The heuristic h(n) is admissible (never overestimates the true remaining distance) and the road network is static.
- Constraints: The robot moves only along existing roads and must find the path minimising total travel cost g(n).

3. AI Technique Selection

A* Search (Informed Search combining path cost and heuristic estimate).

4. Justification of Algorithm

A* evaluates nodes using f(n) = g(n) + h(n), balancing the actual cost travelled so far with the estimated cost remaining. Unlike Greedy Best-First Search, which ignores g(n) and can return a suboptimal route, A* is guaranteed to find the optimal path when h(n) is admissible. Unlike Uniform-Cost Search (which ignores the heuristic entirely), A* uses h(n) to focus the search toward the goal, expanding far fewer nodes — making it the right choice for cost-optimal robot route planning.

5. State Space Representation

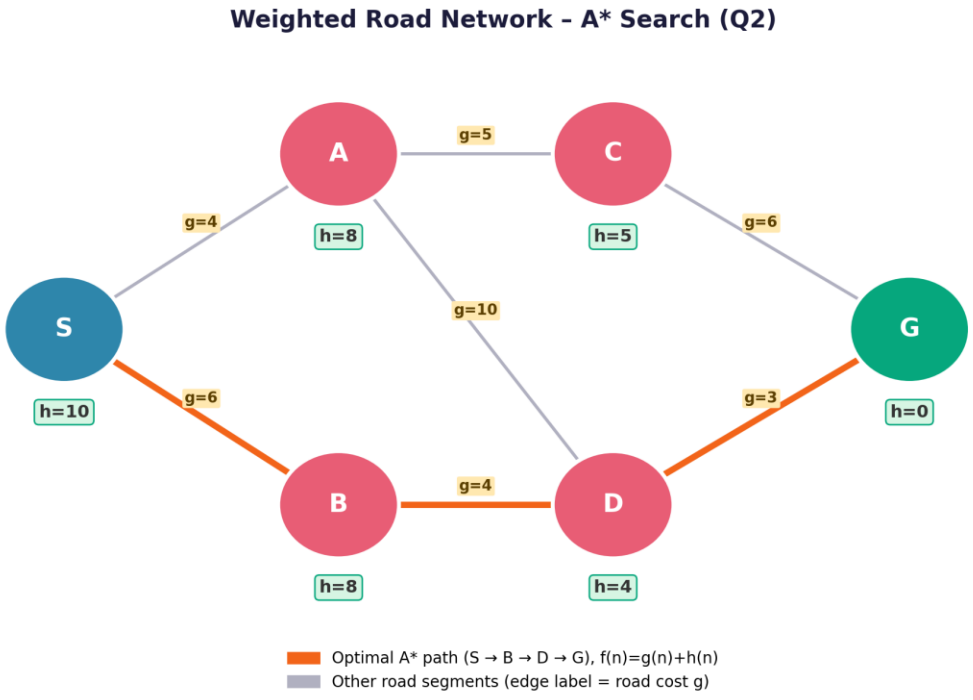

Fig. 2 — Weighted road network with edge costs g and heuristic values h(n); the orange path is the optimal A route.*

6. Variables, Domains and Constraints (CSP)

Not Applicable — this is a graph-search problem, not a CSP.

7. Heuristic Function

h(n) = straight-line distance estimate from node n to the customer location G (values shown under each node in Fig. 2). This heuristic is admissible because it never exceeds the true remaining road distance, which is what allows A* to guarantee optimality.

8. Algorithm Design / Pseudocode

```
function A-STAR(start, goal):
    frontier <- PriorityQueue ordered by f(n) = g(n) + h(n)
    g[start] <- 0
    frontier.push(start, h(start))
    while frontier is not empty:
        n <- frontier.pop_min_f()
        if n == goal: return RECONSTRUCT-PATH(n)
        for each neighbour m of n with edge cost c(n, m):
            tentative_g <- g[n] + c(n, m)
            if m not visited or tentative_g < g[m]:
                g[m] <- tentative_g
                f <- tentative_g + h(m)
                came_from[m] <- n
                frontier.push(m, f)
    return failure
```

9. Flowchart / Search Tree

Refer to Fig. 2 (Section 5) — the orange edges show the search tree branch that is expanded into the optimal, lowest-cost route.

10. Working / Execution Trace

Node Expanded	g(n)	h(n)	f(n) = g+h	Notes
S	0	10	10	start node
A	4	8	12	lowest f in frontier
B	6	8	14	updates D via B
D (via B)	10	4	14	g improved from 14 (via A) to 10
G (goal)	13	0	13	path S → B → D → G

Optimal path found: S → B → D → G with total cost g(G) = 6 + 4 + 3 = 13, which is lower than the alternative S → A → D → G (cost 4+10+3 = 17) or S → A → C → G (cost 4+5+6 = 15).

11. Complexity Analysis

- Time Complexity: O(b^d) in the worst case (b = branching factor, d = solution depth), but far lower in practice with an informative admissible heuristic.
- Space Complexity: O(b^d) — A* stores every generated node, which is its main practical drawback for very large graphs.
- With an admissible and consistent heuristic, A* is guaranteed to expand the minimum number of nodes needed to prove optimality.

12. Advantages and Limitations

- Advantage: Complete and optimal when h(n) is admissible (and consistent).
- Advantage: More goal-directed and efficient than Uniform-Cost Search.
- Limitation: High memory usage — every expanded node is kept in memory.
- Limitation: Performance is highly dependent on the quality of the heuristic function.

13. Conclusion

A* Search correctly identified S → B → D → G as the optimal, least-cost delivery route (total cost 13), demonstrating the advantage of combining actual path cost with heuristic guidance. For very large road networks, memory-bounded variants such as IDA* or SMA* could be used to reduce memory overhead.

QUESTION 3

Constraint Satisfaction Problem — Exam Timetabling

1. Problem Statement

A university must prepare an examination timetable for six courses without scheduling any two courses that share students at the same time slot. The problem must be modelled as a Constraint Satisfaction Problem (CSP) and solved efficiently.

2. Problem Analysis

- Initial State: No course has been assigned an exam slot.
- Goal State: Every course is assigned exactly one slot such that no two conflicting (student-overlapping) courses share a slot.
- Assumptions: Three exam slots are available; conflicts are known in advance from student course-registration data.
- Constraints: Binary 'not-equal' constraints between every pair of courses that share at least one student.

3. AI Technique Selection

Constraint Satisfaction Problem formulation, solved using Backtracking Search enhanced with the Minimum Remaining Values (MRV) heuristic and Forward Checking.

4. Justification of Algorithm

Timetabling is a natural fit for CSP formulation because it involves discrete variables (courses), finite domains (slots), and explicit pairwise constraints (conflicts). Plain backtracking alone can waste time exploring branches that are doomed to fail. The MRV heuristic selects the most constrained course first, sharply reducing the branching factor, while Forward Checking immediately removes now-invalid values from neighbouring domains, detecting dead ends far earlier than plain backtracking.

5. State Space Representation

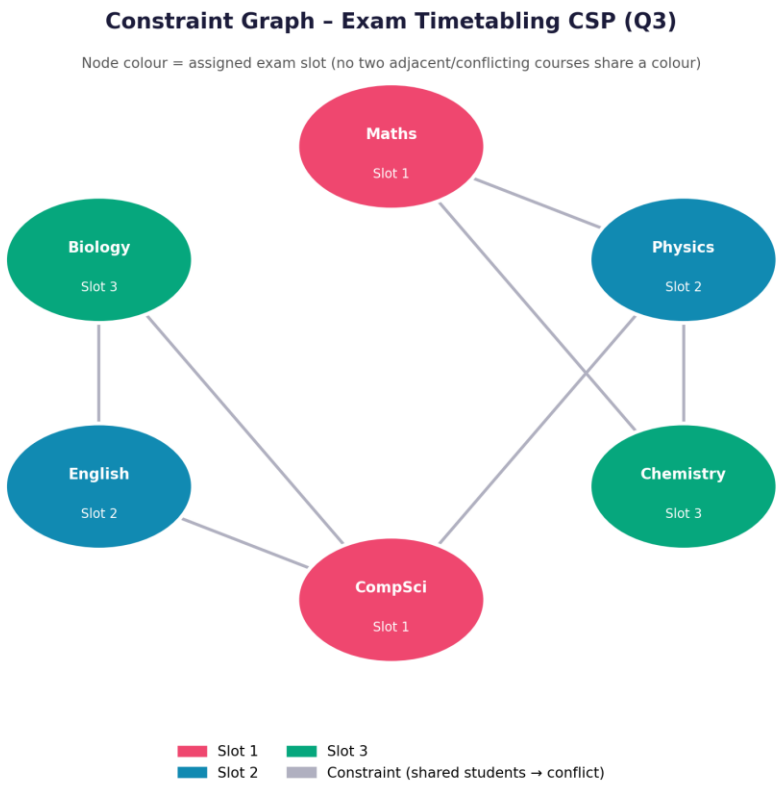

Fig. 3 — Constraint graph for the exam timetable; node colour shows the final assigned slot, edges show student-overlap conflicts.

6. Variables, Domains and Constraints

Variable (Course)	Domain	Conflicts With (Constraints)
Maths	{Slot 1, Slot 2, Slot 3}	Physics, Chemistry
Physics	{Slot 1, Slot 2, Slot 3}	Maths, Chemistry, CompSci
Chemistry	{Slot 1, Slot 2, Slot 3}	Maths, Physics
CompSci	{Slot 1, Slot 2, Slot 3}	Physics, English, Biology
English	{Slot 1, Slot 2, Slot 3}	CompSci, Biology
Biology	{Slot 1, Slot 2, Slot 3}	CompSci, English

7. Heuristic Function

Not a numerical h(n) as in informed search — instead, variable-ordering heuristics are used: MRV (choose the variable with the fewest legal remaining values) and the Degree heuristic (break ties by choosing the variable involved in the most constraints) guide which course is scheduled next.

8. Algorithm Design / Pseudocode

```
function BACKTRACKING-SEARCH(csp):
    return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp):
    if assignment is complete: return assignment
    var <- SELECT-UNASSIGNED-VARIABLE(csp)   // MRV, tie-break by degree
```

```
    for value in ORDER-DOMAIN-VALUES(var, assignment, csp):
        if value is consistent with assignment:
            assignment[var] <- value
            inferences <- FORWARD-CHECKING(csp, var, value)
            if inferences != failure:
                result <- BACKTRACK(assignment, csp)
                if result != failure: return result
            remove value, undo inferences
        assignment.remove(var)
    return failure
```

9. Flowchart / Constraint Graph

Refer to Fig. 3 (Section 5) for the constraint graph — the final node colours represent a valid 3-colouring, i.e. a conflict-free timetable.

10. Working / Execution Trace

Step	Variable (MRV)	Domain Before	Assigned	Pruned by Forward Checking
1	Maths	{S1,S2,S3}	S1	Physics→{S2,S3}, Chemistry→{S2,S3}
2	Physics	{S2,S3}	S2	Chemistry→{S3}, CompSci→{S1,S3}
3	Chemistry	{S3}	S3	no unassigned neighbours left
4	CompSci	{S1,S3}	S1	English→{S2,S3}, Biology→{S2,S3}
5	English	{S2,S3}	S2	Biology→{S3}
6	Biology	{S3}	S3	assignment complete, no conflicts

The MRV + Forward Checking search reached a complete, conflict-free assignment in exactly 6 steps with zero backtracking.

11. Complexity Analysis

- Time Complexity: O(d^n) worst case for plain backtracking (n = variables, d = domain size); MRV + Forward Checking greatly reduces the practical branching factor.
- Space Complexity: O(n) for the assignment stack plus O(n·d) to store pruned domains.
- For this instance (6 variables, 3-value domains), the enhanced search solved the CSP with no backtracking at all.

12. Advantages and Limitations

- Advantage: Backtracking search is complete — it finds a solution whenever one exists.
- Advantage: Forward Checking detects failure early, avoiding wasted exploration of doomed branches.
- Advantage: MRV substantially reduces the effective branching factor by tackling the hardest variables first.
- Limitation: Dense constraint graphs can still require heavy backtracking; stronger consistency techniques (e.g., AC-3) may be needed for larger timetables.

13. Conclusion

Modelling the exam timetable as a CSP and solving it with MRV-guided backtracking plus Forward Checking produced a valid, conflict-free 3-slot schedule with no backtracking required. For larger universities with many more courses, adding AC-3 preprocessing or a min-conflicts local-search repair step would further improve scalability.

QUESTION 4

Minimax & Alpha–Beta Pruning — Game Tree Analysis

1. Problem Statement

Given a two-player, zero-sum game tree with known terminal utility values, determine the best move for the MAX player using the Minimax algorithm, then apply Alpha–Beta Pruning to the same tree and compare the number of nodes evaluated before and after pruning.

2. Problem Analysis

- Initial State: Root of the game tree, MAX to move.
- Goal/Terminal States: The 8 leaf nodes, each holding a fixed utility value.
- Assumptions: Perfect information, deterministic moves, both players play optimally.
- Constraints: MAX always chooses the child with the highest value; MIN always chooses the child with the lowest value.

3. AI Technique Selection

Minimax algorithm, optimized with Alpha–Beta Pruning.

4. Justification of Algorithm

Minimax guarantees optimal play against a perfectly rational opponent in deterministic, perfect-information games by exploring the entire game tree. Alpha–Beta Pruning computes the exact same result while skipping branches that can be proven irrelevant to the final decision — making it strictly better than plain Minimax whenever speed matters, without ever changing the chosen move.

5. State Space Representation

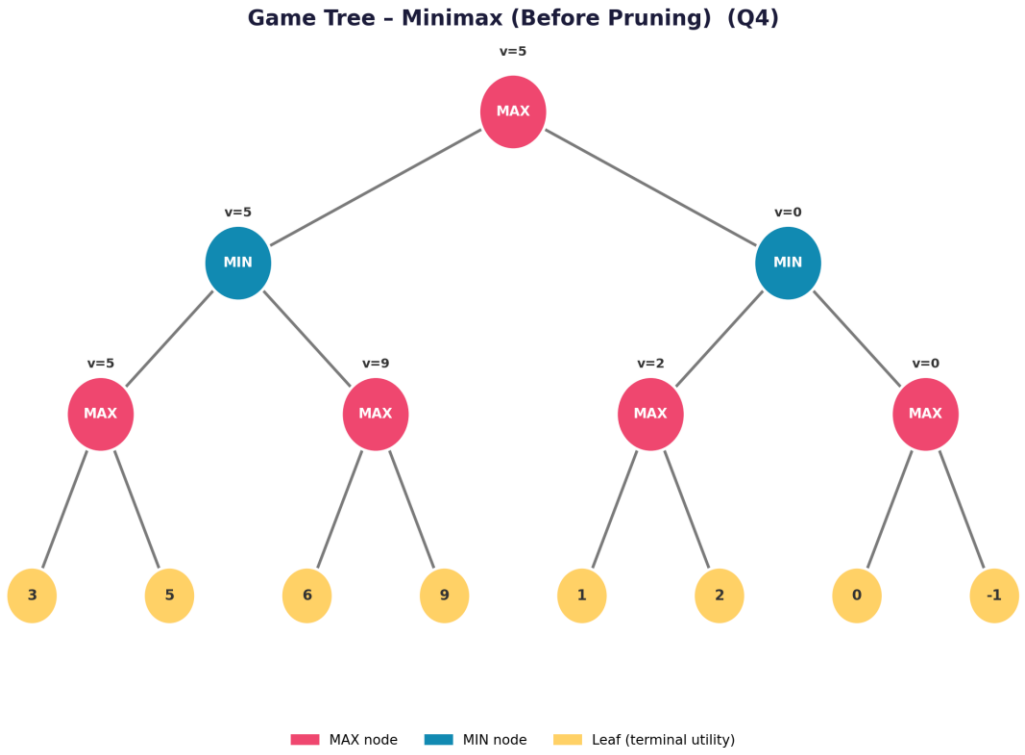

Fig. 4a — Full game tree evaluated by plain Minimax (all 8 leaves visited).

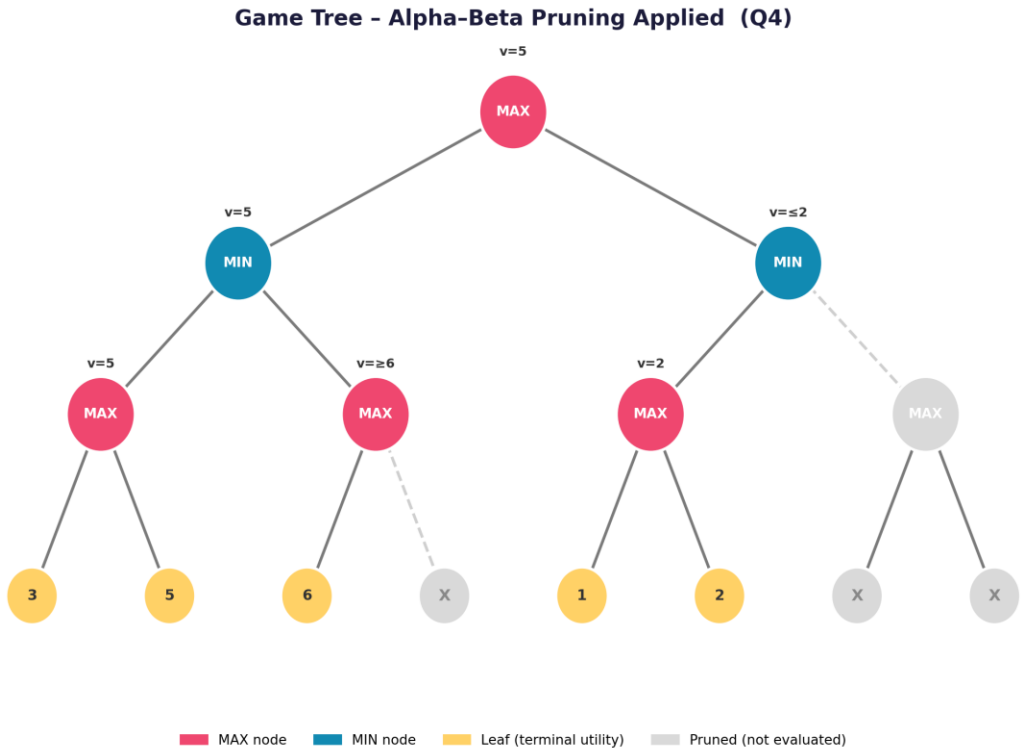

Fig. 4b — Same tree under Alpha–Beta Pruning; greyed-out nodes marked X are never evaluated.

6. Variables, Domains and Constraints (CSP)

Not Applicable — this is an adversarial game-tree search problem, not a CSP.

7. Heuristic Function

Not Applicable in this instance — the tree is searched to full depth, so the leaf utility values are exact terminal outcomes rather than heuristic estimates. (For a depth-limited search on a larger game, a static evaluation function would substitute for exact utilities.)

8. Algorithm Design / Pseudocode

```
function MINIMAX(node, maximizingPlayer):
    if node is terminal: return utility(node)
    if maximizingPlayer:
        value <- -infinity
        for child in children(node):
            value <- max(value, MINIMAX(child, False))
        return value
    else:
        value <- +infinity
        for child in children(node):
            value <- min(value, MINIMAX(child, True))
        return value

function ALPHA-BETA(node, alpha, beta, maximizingPlayer):
    if node is terminal: return utility(node)
    if maximizingPlayer:
        value <- -infinity
        for child in children(node):
            value <- max(value, ALPHA-BETA(child, alpha, beta, False))
```

```
            alpha <- max(alpha, value)
            if alpha >= beta: break   // beta cut-off (prune)
        return value
    else:
        value <- +infinity
        for child in children(node):
            value <- min(value, ALPHA-BETA(child, alpha, beta, True))
            beta <- min(beta, value)
            if beta <= alpha: break   // alpha cut-off (prune)
        return value
```

9. Flowchart / Game Tree Diagram

Refer to Fig. 4a and Fig. 4b (Section 5) for the full tree and the pruned tree respectively.

10. Working / Execution Trace

Bottom-up Minimax evaluation (Fig. 4a):

- MAX nodes at depth 3: E1 = max(3,5) = 5, E2 = max(6,9) = 9, E3 = max(1,2) = 2, E4 = max(0,-1) = 0
- MIN nodes at depth 2: B = min(E1,E2) = min(5,9) = 5, C = min(E3,E4) = min(2,0) = 0
- Root (MAX): value = max(B,C) = max(5,0) = 5 → best move for MAX leads into the B subtree.

Alpha–Beta trace (Fig. 4b):

- Left subtree (B): E1 fully evaluated = 5, so B's β becomes 5. In E2, leaf '6' is found and 6 ≥ β(5), triggering a beta cut-off — the second leaf '9' under E2 is pruned (never evaluated). B resolves to 5, and root's α becomes 5.
- Right subtree (C): E3 fully evaluated = 2, so C's β becomes 2. Since C's β(2) ≤ root's α(5), an alpha cut-off occurs — the entire E4 subtree (leaves '0' and '-1') is pruned without being visited.
- Root value = max(5, ≤2) = 5 — identical to the unpruned result.

11. Complexity Analysis

Metric	Plain Minimax	With Alpha–Beta Pruning
Leaves evaluated	8 of 8	5 of 8
Total nodes visited	15	11
Final Minimax value	5	5 (unchanged)
Time Complexity	O(b^d)	O(b^(d/2)) best case with good ordering

12. Advantages and Limitations

- Advantage: Alpha–Beta Pruning never changes the final decision — correctness is fully preserved.
- Advantage: Up to 4 fewer nodes (27% reduction) were visited in this small example; savings grow rapidly with tree size.
- Limitation: The amount of pruning depends heavily on move ordering — a poor ordering degrades to the same cost as plain Minimax.
- Limitation: Both algorithms assume the full tree can be searched; large games (e.g., chess) require a depth cut-off with a heuristic evaluation function.

13. Conclusion

Alpha–Beta Pruning reached the identical optimal value (5) as plain Minimax while skipping 3 of the 8 leaf evaluations, confirming that it improves efficiency without sacrificing correctness. Applying move-ordering heuristics (e.g., evaluating the most promising child first) would push pruning closer to the theoretical O(b^(d/2)) best case.

QUESTION 5

Online Search Agent — Dynamic Warehouse Robot

1. Problem Statement

A warehouse robot must travel to a target location, but the environment is dynamic — obstacles may appear unexpectedly and are not known in advance. Design an intelligent search strategy that lets the robot adapt in real time rather than relying on a single, precomputed offline path.

2. Problem Analysis

- Initial State: Robot's current warehouse cell/location.
- Goal State: Target shelf / delivery location.
- Assumptions: The environment is only partially known in advance; new obstacles can appear at runtime and must be sensed as they occur.
- Constraints: The robot can only act on locally sensed information — it cannot look ahead beyond its immediate neighbourhood.

3. AI Technique Selection

An Online Search Agent built around Local Search (Hill-Climbing with a Random-Restart escape strategy), since offline algorithms such as A* assume a fully known, static map.

4. Justification of Algorithm

Online search interleaves computation with action: the robot senses, decides its next move, acts, and re-senses — which is essential when the map is not fully known beforehand. Hill-climbing local search requires only the current state in memory and reacts instantly to newly sensed obstacles, unlike A* or Greedy Best-First Search, which would need to recompute a full path over the whole map every time an obstacle appears. This makes it far better suited to a resource-constrained, real-time robot operating in a changing environment.

5. State Space Representation / Flowchart

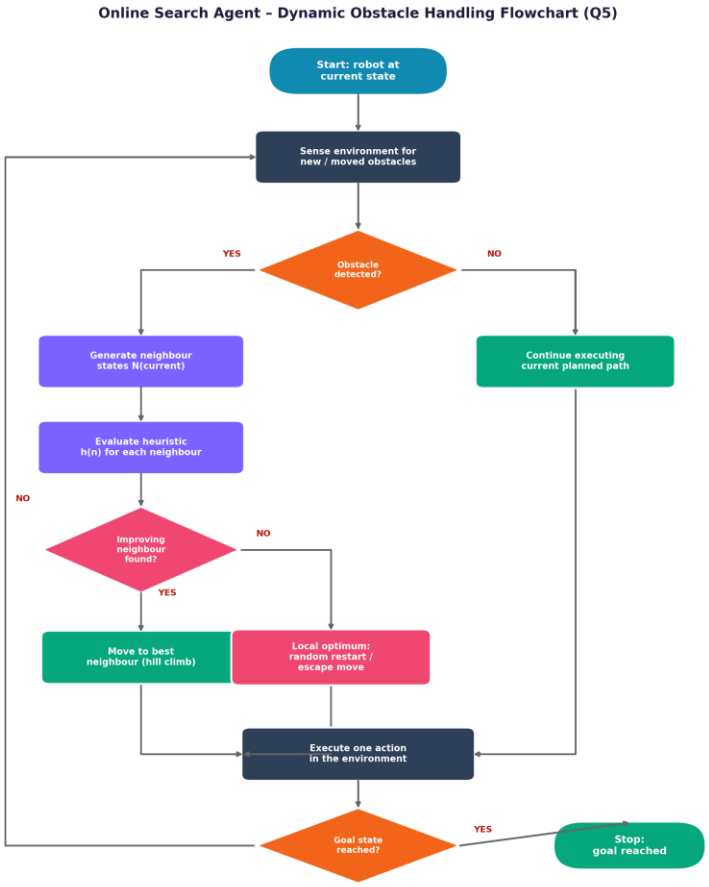

Fig. 5 — Flowchart of the online search agent reacting to dynamic obstacles using local (hill-climbing) search.

6. Variables, Domains and Constraints (CSP)

Not Applicable — this is a real-time search-agent problem, not a CSP.

7. Heuristic Function

h(n) = Manhattan distance from the robot's current grid cell to the goal cell. At every sensing cycle, the agent evaluates h(n) for each free neighbouring cell and greedily moves toward the neighbour with the lowest value, i.e. the one that appears closest to the goal.

8. Algorithm Design / Pseudocode

```
function ONLINE-DYNAMIC-SEARCH-AGENT(goal):
    current <- sense_initial_state()
    while current != goal:
        obstacles <- sense_environment()
        if planned_step_is_blocked(current, obstacles):
            neighbours <- get_free_neighbours(current)
            best <- argmin(neighbours, key = h)
            if h(best) < h(current):
                current <- move_to(best)          // hill-climb step
            else:
                current <- random_restart(neighbours)  // escape local optimum
        else:
            current <- move_to(next_step_on_plan)
    return success
```

9. Flowchart / Algorithm Diagram

Refer to Fig. 5 (Section 5) for the complete sense–decide–act cycle, including the local-optimum escape (random-restart) branch.

10. Working / Execution Trace

While moving along its planned route toward the goal shelf, the robot's sensors detect an unexpected pallet blocking the next planned step. It generates the set of free neighbouring cells {North, East, South, West} and computes h(n) (Manhattan distance to goal) for each. The East cell has the lowest heuristic value and is chosen, since it improves on the current cell's value — the robot moves there and resumes sensing. If the robot later enters a pocket where every free neighbour has an equal or worse heuristic value than the current cell (a local optimum), it triggers a random restart, jumping to a randomly chosen reachable free cell to escape the stall and continue progressing toward the goal.

11. Complexity Analysis

- Time Complexity: O(b) per decision step, where b is the number of neighbouring cells (e.g., 4 in a grid) — far cheaper than replanning a full path.
- Space Complexity: O(1) — only the current state needs to be stored, unlike A*'s O(b^d) frontier storage.
- Random restarts add expected multiplicative overhead in the worst case but keep the memory footprint constant.

12. Advantages and Limitations

- Advantage: Extremely memory-efficient — ideal for embedded/onboard robot hardware.
- Advantage: Reacts instantly to newly sensed obstacles without recomputing a global path.
- Limitation: No completeness or optimality guarantee; can stall at local optima or plateaus (mitigated here by random restart).
- Limitation: May take a longer overall route than an optimal offline solution computed with full map knowledge.

13. Conclusion

The online, sensor-driven local-search agent successfully adapted to an unexpected obstacle using minimal memory and computation, at the cost of a guaranteed-optimal route. A natural extension is Learning Real-Time A* (LRTA*), which keeps the same reactive, online property while learning improved heuristic estimates over repeated traversals, producing progressively shorter paths on future runs through the same warehouse.